动态规划

定义

动态规划是一种通过将复杂问题分解为重叠子问题，并利用子问题的解来高效求解原问题的算法设计方法。本质是将子问题的解存储起来，避免重复计算，从而提高效率。

- 阶段 阶段变量 k
- 状态 状态变量 sk ,状态集合 Sk
- 决策 xk(sk) ,决策集合 Dk(sk)
- 策略 p1n={x1(s1),x2(s2),…,xn(sn)}
- k 子过程策略 pkn={xk(sk),xk+1(sk+1),…,xn(sn)} 。
- 状态转移方程 sk+1=T(sk,xk)=xk (sk)
- 阶段效应 (利润、成本、距离) rk(sk,xk)
- 最优指标函数 第 k 阶段的状态 sk，当采取最优策略(xk, xk+1,… xn)后，从阶段 k 到阶段 n 可获得的总效应，称为最优指标函数，记为 fk(sk)。

- ## 性质

最优子结构（Optimal Substructure）

问题的最优解包含其子问题的最优解。即：若原问题的最优解在某一决策点选择了某个状态，则该状态对应的子问题的解也必须是最优的。

无后效性（No Aftereffect）

一旦当前状态确定，其后续决策不依赖于过去的状态，仅由当前状态决定。即：历史信息已被当前状态完全概括，未来的决策无需考虑之前的具体路径。

马尔可夫链（Markov Chain）

定义

马尔科夫链是一种具有马尔科夫性质（无记忆性）的随机过程，描述系统在状态空间中随时间转移的概率模型。其核心思想是：系统未来的状态仅依赖于当前状态，与过去的历史状态无关。

数学定义如下：

设$\{X_t, t \in T\}$为随机过程，状态空间为S（离散或可数集合）。若对任意时刻$t \geq 0$和任意状态$i, j, i_0, i_1, \ldots, i_{t-1} \in S$，有

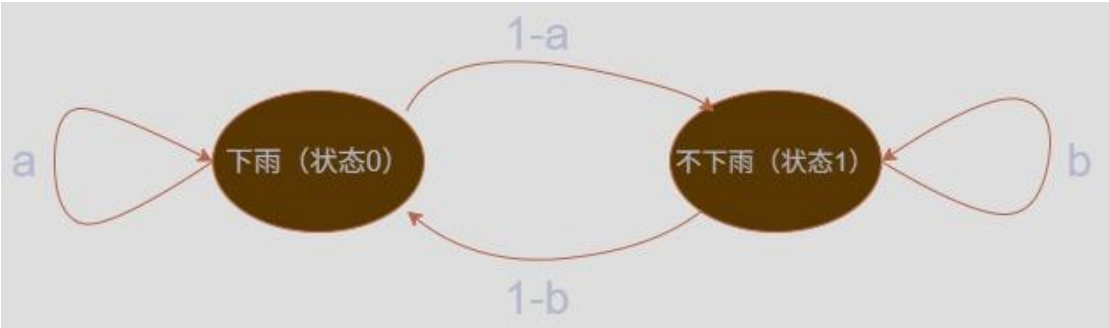
$$P(X_{t+1}=j \mid X_t=i, X_{t-1}=i_{t-1}, \ldots, X_0=i_0)=P(X_{t+1}=j \mid X_t=i),$$

则称该过程为**马尔科夫链**。

转移矩阵可写为$P=\begin{pmatrix} P_{00} & P_{01} & P_{00} & \cdots \\ P_{10} & P_{11} & P_{00} & \cdots \\ \vdots & \vdots & \vdots & \\ P_{i0} & P_{i1} & P_{i2} & \cdots \end{pmatrix}$

例：假设明天下雨的机会只依赖于前一天的天气条件，即今天是否下雨,而不依赖过去的天气条件.再假设如果今天下雨,那么明天下雨的概率为 a;如果今天没有下雨,那么明天下雨的概率为 b.

如果下雨,我们假定过程在状态 0;如果不下雨,我们假定过程在状态 1.那么上面的内容是一个两个状态的马尔可夫链,其转移概率矩阵给定为：$P=\begin{pmatrix} a & 1-a \\ b & 1-b \end{pmatrix}$



性质

1.马尔科夫性（无记忆性）

未来状态的概率分布仅由当前状态决定，与历史路径无关。

数学表达：$P(X_{t+n}=j \mid X_t=i, X_{t-1}=i_{t-1},\ldots,X_0=i_0)=P(X_{t+n}=j \mid X_t=i)$.

2.状态空间与转移概率矩阵

状态空间 S：离散（如有限个状态 $\{1,2,\ldots,N\}$）或可数无限（如自然数集）。

转移概率矩阵 $\mathbf{P}=(P_{i,j})$：

$P_{i,j}\geq 0$，表示从状态 i 转移到状态 j 的概率；

每行和为 1，即$\sum_{j\in S}P_{i,j}=1$，满足概率归一性。

可以看到，马尔可夫链与动态规划都具有相似的性质，使用动态规划可以有效地应用在马尔可夫链相关问题中。

马尔可夫决策过程（MDP）

组成要素

状态空间：系统所有可能状态的集合。例如，在一个机器人导航问题中，状态可以是机器人在地图上的不同位置。

动作空间：在每个状态下，可供选择的动作集合。比如机器人可以有前进、后退、左转、右转等动作。

转移概率：给定当前状态和动作，系统转移到下一个状态的概率分布。例如，机器人在当前位置执行前进动作后，到达下一个位置的概率。

奖励函数：当系统从一个状态转移到另一个状态时，给予的奖励值。奖励用于衡量不同决策序列的好坏，引导智能体做出最优决策。例如，机器人到达目标位置时会得到一个正奖励，而撞到障碍物则会得到一个负奖励。

同时奖励分为即时奖励和长期奖励。即时奖励表示从 st（当前状态）转换到 st+1（下一状态）的奖励，即时奖励与状态转移相伴随，只要发生状态转移，就一定会产生对应的即时奖励。长期奖励公式为：

$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + ... = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$ γ为衰减系数（折扣因子），表示即时奖励在当前状态的折扣值。长期奖励（也称回报）是与状态转移链相伴随，不同的状态转移链对应不同的长期奖励。

长期奖励G_t同时可以也写成：

$$\begin{aligned} Gt &= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots, \\ &= R_{t+1} + \gamma (R_{t+2} + \gamma R_{t+3} + \ldots), \\ &= R_{t+1} + \gamma G_{t+1}. \end{aligned}$$

由此可以得到状态值函数的**贝尔曼方程**：

$$\begin{aligned} v_\pi(s) &= E_\pi[Gt \ / \ St = s] \\ &= E_\pi[Rt+1 + \gamma Gt+1 \ / \ St = s] \\ &= \sum_a \pi(a \mid s) \sum_{s',r} p(s',r \ / \ s,a)[r + \gamma v_\pi(s')], \ \forall s \in S. \end{aligned}$$

它表示从特定状态出发，按照某种策略进行决策所能得到的回报期望值。

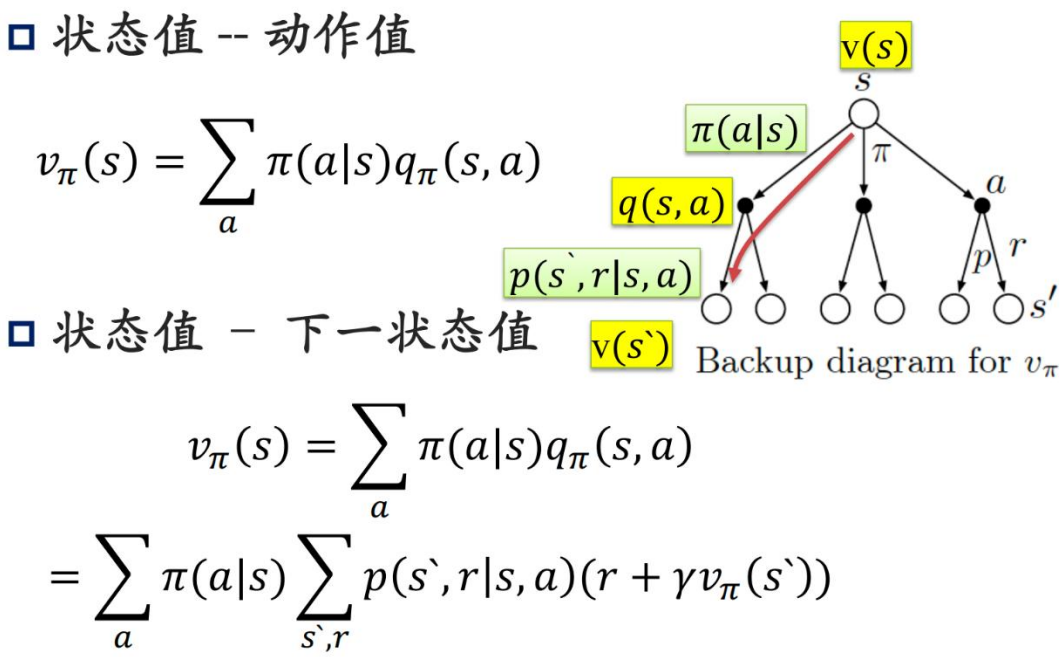
推导过程如下：

可以将G_t拆成两部分

$$
\begin{aligned}
v_\pi(s) &= E[G_t \ / S_t = s] \\
&= E[R_{t+1} + \gamma G_{t+1} \ / S_t = s] \\
&= E[R_{t+1} \ / S_t = s] + \gamma E[G_{t+1} \ / S_t = s]
\end{aligned}
$$

也就是即时奖励与长期奖励，对其使用全概率公式与马尔可夫性质即可得到：

$$
\begin{aligned}
v_\pi(s) &= E[R_{t+1} \mid S_{t=s}] + \gamma E[G_{t+1} \mid S_{t=s}] \\
&= \sum_a \pi(a \mid s) \sum_r p(r \mid s,a) r + \gamma \sum_a \pi(a \mid s) \sum_{s'} p(s' \mid s,a) v_\pi(s') \\
&= \sum_a \pi(a \mid s) \sum_{s',r} p(s',r \mid s,a)[r + \gamma v_\pi(s')], \ \forall s \in S.
\end{aligned}
$$



将贝尔曼公式拆成两项之和的形式

$$
v_\pi(s) = r_\pi(s) + \gamma \sum_{s'} p_\pi(s' \mid s) v_\pi(s')
$$

其中 $r_\pi(s) = \sum_a \pi(a \mid s) \sum_r p(r \mid s,a) r, \qquad p_\pi(s' \mid s) = \sum_a \pi(a \mid s) p_\pi(s' \mid s,a)$

假设状态为 s_i（i = 1，2……，n）,对于状态 s_i,贝尔曼方程为：

$$
v_\pi(s_i) = r_\pi(s_i) + \gamma \sum_{s_j} p_\pi(s_j \mid s_i) v_\pi(s_j)
$$

写成矩阵形式，有：

$v_\pi = r_\pi + \gamma P_\pi v_\pi$

其中$v_\pi = [v_\pi(s_1),\ldots,v_\pi(s_n)]^T \in \mathbb{R}^n$

$r_\pi = [r_\pi(s_1),\ldots,r_\pi(s_n)]^T \in \mathbb{R}^n$

$P_\pi \in \mathbb{R}^{n\times n}$,其中$[P_\pi]_{ij} = p_\pi(s_j \mid s_i)$为状态转移矩阵

对给定的策略，要得到各状态下的状态值，

对矩阵求逆易得$v_\pi =$（$E - \gamma P_\pi$ ）$^{-1} r_\pi$

但矩阵的求逆是 O（n^3）的复杂度，当矩阵很大时，求解效率很低，往往采用其他方法求解。

在寻找最大值时，贝尔曼方程就转换成最大值形式

即最优状态值函数($V^*(S)$)

$$V*(s) = \max_\pi \left\{ \sum_a \pi(a \mid s) \sum_{s',r} p(s',r \mid s,a)\left[r + \gamma v_\pi(s')\right] \right\},\ \forall s \in S.$$

通过迭代更新价值函数来逐步逼近最优价值函数，并根据最优价值函数选择最优策略。

1. 初始化价值函数：
   对价值函数进行初始化。通常将价值函数的初始估计设为 0 或者随机值。
2. 迭代更新价值函数：
   将每个状态都通过贝尔曼最优方程进行价值函数的更新。贝尔曼最优方程表示了最优值函数之间的关系，用来更新值函数的估计。价值函数的公式更新为$V_{k+1}(s) = \max_s \sum_{s'} p(s' \mid s,a)\left[r(s,a,s') + \gamma V_K(s')\right]$。
3. 策略改进：
   在每次值函数更新后，根据新的价值函数来进行策略改进，即选择能够使

价值函数最大化的动作作为最优策略。最优策略的选择通过
$\pi(s) = arg\max_{s} \sum_{s'} p(s' \mid s,a)\,[r(s,a,s') + \gamma V(s')]$ 实现。

价值迭代算法保证在有限步内收敛到最优价值函数，由于价值函数每次更新都会使价值函数逼近最优价值函数，且在有限状态空间内，价值函数一定会收敛。最后，当价值函数不再发生变化时，即最优价值函数。

通过贝尔曼方程（Bellman Equation）将 MDP 问题分解为子问题，可使用动态规划，利用最优子结构和重叠子问题特性高效求解。

对于时间复杂度

定义

状态数量（N）：问题的状态空间大小。

每个状态的动作数量（A）：每个状态下可选择的动作数。

状态转移分支数（S）：每个动作可能转移到的下一状态数量（例如随机环境中的多分支转移）。

迭代次数（K）：算法收敛所需的迭代轮次（通常与折扣因子γ和收敛阈值相关）。

时间复杂度为

单次迭代的时间复杂度：$O(N \times A \times S)$

每个状态需遍历所有动作，每个动作需计算所有可能的转移分支。

总时间复杂度：$O(K \times N \times A \times S)$，小于矩阵求逆的 O（$N^3$）

一般的代码

```
def value_iteration(states, actions, transition, reward, gamma, threshold=1e-6):
    # 初始化值函数 V(s)
    V = {s: 0 for s in states}   # 终止状态的值可预设

    while True:
        delta = 0
        new_V = {}   # 临时存储新值

        # 遍历所有状态
        for s in states:
            if is_terminal(s):   # 终止状态不更新
                new_V[s] = reward(s)
                continue

            max_value = -inf
            # 遍历所有可能的动作
            for a in actions(s):   # 可能需要根据状态限制可用动作
                # 计算该动作的期望价值（考虑状态转移）
```

```
                    expected_value = 0
                    for s_prime in transition(s, a):   # 遍历所有可能的下一状态
                        prob = transition_prob(s, a, s_prime)    # 转移概率
P(s'|s,a)
                        r = reward(s, a, s_prime)                # 即时奖励
R(s,a,s')
                        expected_value += prob * (r + gamma * V[s_prime])

                    # 保留最大值
                    if expected_value > max_value:
                        max_value = expected_value

                # 更新值函数
                new_V[s] = max_value
                delta = max(delta, abs(new_V[s] - V[s]))

            # 同步更新值函数
            V = new_V.copy()

            # 检查收敛
            if delta < threshold:
                break

    # 提取最优策略
    policy = {}
    for s in states:
        if is_terminal(s):
            policy[s] = None
            continue

        best_action = None
        best_value = -inf
        for a in actions(s):
            expected_value = sum(
                transition_prob(s, a, s_prime) * (reward(s, a, s_prime) + gamma
* V[s_prime])
                for s_prime in transition(s, a)
            )
            if expected_value > best_value:
                best_value = expected_value
                best_action = a
        policy[s] = best_action

    return V, policy
```

在金融领域，马尔可夫决策过程及贝尔曼方程可被用于期权定价、投资组合优化、风险管理、公司财务决策等，除此之外，还被应用于强化学习、控制理论等其他问题中。

例子：

假设一个简单的金融交易场景，状态空间 S 包含两个状态：s1 表示股票价格上涨，s2 表示股票价格下跌；动作空间 A 包含两个动作：a1 表示买入，a2 表示卖出。

状态转移概率如下：

P(s1｜s1, a1)=0.7， P(s2｜s1,a1)=0.3

P(s1|s1,a2)=0.3,P(s2 ｜s1 ,a2)=0.7

P(s1｜s2,a1)=0.3， P(s2｜s2,a1)=0.7

P(s1｜s2,a2)=0.7， P(s2｜s2,a)=0.3

$$P=\begin{cases}\begin{pmatrix}0.7 & 0.3\\ 0.3 & 0.7\end{pmatrix}\\ \begin{pmatrix}0.3 & 0.7\\ 0.7 & 0.3\end{pmatrix}\end{cases}$$

也就是说，在状态 s1 即价格上涨的情况下，采取动作 a1 即买入，状态 s1 有 0.3 的概率转换为 s2,0.7 的概率不变……

奖励函数如下：

R(s1,a1)=10,R(s1,a2)=−5

R(s2,a1)=−10，R(s2,a2)=5

折扣因子γ=0.9

假设当前策略π是在状态 s1 下以概率 0.6 买入，以概率 0.4 卖出；在状态 s2 下以概率 0.3 买入，以概率 0.7 卖出，通过贝尔曼方程可以得到各阶段状态价值函数，并通过迭代，可以得到各情况下的最优决策状态：

上涨（s1）：动作 买入（a1）

状态 下跌（s2）：动作 卖出（a2）

对应状态值函数：V(s1) = 86.50, V(s2) = 81.50

```
import numpy as np

# 定义参数
gamma = 0.9
max_iterations = 100
epsilon = 1e-6

# 奖励函数
R = {
    (0, 0): 10,
    (0, 1): -5,
    (1, 0): -10,
    (1, 1): 5
}

# 转移概率
P = {
    (0, 0): np.array([0.7, 0.3]),
    (0, 1): np.array([0.3, 0.7]),
    (1, 0): np.array([0.3, 0.7]),
    (1, 1): np.array([0.7, 0.3])
}

# 初始化策略（示例初始化，后续会改进）
pi = np.zeros((2, 2))
pi[0, 0] = 1.0  # s1 初始选 a1（买入）
pi[1, 0] = 1.0  # s2 初始选 a1（买入）

while True:
    # 策略评估：计算当前策略下的值函数 V
    V = np.zeros(2)
    while True:
        V_new = np.zeros(2)
        for s in range(2):
            for a in range(2):
                # 基于当前策略 pi 计算期望回报
                V_new[s] += pi[s, a] * (R[(s, a)] + gamma * np.dot(P[(s,
a)], V))
        if np.max(np.abs(V_new - V)) < epsilon:
            V = V_new
            break
        V = V_new
```

```python
    # 策略改进：更新策略
    policy_stable = True
    new_pi = np.zeros((2, 2))
    for s in range(2):
        q_values = []
        for a in range(2):
            # 计算每个动作的 Q 值
            q = R[(s, a)] + gamma * np.dot(P[(s, a)], V)
            q_values.append(q)
        best_a = np.argmax(q_values)  # 选 Q 值最大的动作
        new_pi[s, best_a] = 1.0
        if not np.array_equal(pi[s], new_pi[s]):
            policy_stable = False
    pi = new_pi

    if policy_stable:
        break

# 输出结果
print("最优策略：")
for s in range(2):
    action_desc = "买入（a1）" if np.argmax(pi[s]) == 0 else "卖出（a2）"
    state_desc = "上涨（s1）" if s == 0 else "下跌（s2）"
    print(f"状态 {state_desc}：动作 {action_desc}")
print(f"对应状态值函数：V(s1) = {V[0]:.2f}, V(s2) = {V[1]:.2f}")
```

例子

假设投资者持有一份**美式看跌期权**，标的资产当前价格为 $S0=100$，行权价为 $K=95$，期权有效期 3 个月（按每月一步离散化）。标的资产价格每月变化遵循**二叉树模型**：

每期价格上涨概率为 $p=0.5$，涨幅为 $u=1.05$（即 $St+1=St\times u$）。

每期价格下跌概率为 $1-p=0.5$，跌幅为 $d=0.96$（即 $St+1=St\times d$）。

无风险利率为每月 $r=1\%$。

目标：通过动态规划确定每期的最优决策（持有或行权），以最大化期权的期望收益现值。

动态规划建模

1. 状态变量

当前时间 t：$t=0,1,2,3\cdots\cdots,T$。

标的资产价格 St：每个时间点的可能价格由二叉树路径生成。

2. 动作空间

行权（Exercise）：立即获得收益 max(K−St,0)。

持有（Hold）：继续等待至下一期。

3. 贝尔曼方程

定义值函数 V(t,St)为在时间 t、价格 St 时的期权价值：

$$V(t,S_t)=\max\left(\max(K-S_t,0),\frac{1}{1+r}\left[p\cdot V(t+1,S_t\cdot u)+(1-p)\cdot V(t+1,S_t\cdot d)\right]\right)$$

值迭代更新公式

$$V_{k+1}(s)=\max_a\left[\sum_{s'}P\left(s'|s,a\right)\left(R(s,a,s')+\gamma V_k(s')\right)\right]$$

代码实现：

```
import numpy as np

def american_put_decision(T, S0, K, r, sigma, T_total):
    # 参数计算
    dt = T_total / T
    u = np.exp(sigma * np.sqrt(dt))
    d = 1 / u
    p = (np.exp(r * dt) - d) / (u - d)
    discount = np.exp(-r * dt)

    # 初始化价格树、值树和决策树
    price_tree = np.zeros((T+1, T+1))
    value_tree = np.zeros((T+1, T+1))
    decision_tree = np.empty((T+1, T+1), dtype=object)   # 存储"Exercise"或"Hold"

    # 生成标的资产价格树
    for t in range(T+1):
        for i in range(t+1):
            price_tree[t, i] = S0 * (u ** (t - i)) * (d ** i)

    # 终止条件（到期日）
    value_tree[T, :] = np.maximum(K - price_tree[T, :], 0)
    decision_tree[T, :] = "Exercise"   # 到期必须行权

    # 逆向递归计算值和决策
    for t in range(T-1, -1, -1):
        for i in range(t+1):
            current_price = price_tree[t, i]
```

```
            exercise_value = max(K - current_price, 0)

            # 计算持有现值
            hold_value = discount * (p * value_tree[t+1, i] + (1-p) * value_tree[t+1, i+1])

            # 记录最优决策
            if exercise_value > hold_value:
                value_tree[t, i] = exercise_value
                decision_tree[t, i] = "Exercise"
            else:
                value_tree[t, i] = hold_value
                decision_tree[t, i] = "Hold"

    return price_tree, value_tree, decision_tree

# 参数设定（3 个月，分 3 期）
T = 3
S0 = 100
K = 95
r = 0.01    # 月利率 1%
sigma = 0.2
T_total = 0.25    # 3 个月=0.25 年

price_tree, value_tree, decision_tree = american_put_decision(T, S0, K, r, sigma, T_total)

# 打印决策表
print("时间 | 价格          | 期权价值 | 最优决策")
print("-----|-----------|----------|----------")
for t in range(T+1):
    for i in range(t+1):
        price = price_tree[t, i]
        value = value_tree[t, i]
        decision = decision_tree[t, i]
        print(f"{t:2d}    | {price:7.2f}    | {value:6.2f}    | {decision}")
    print("-----|-----------|----------|----------")
```

结果为：

```
时间 | 价格          | 期权价值 | 最优决策
-----|-----------|----------|----------
  0    |    100.00    |      1.67    | Hold
-----|-----------|----------|----------
  1    |    105.94    |      0.16    | Hold
  1    |     94.39    |      3.14    | Hold
-----|-----------|----------|----------
```

```
  2   |   112.24   |     0.00   | Hold
  2   |   100.00   |     0.31   | Hold
  2   |    89.09   |     5.91   | Exercise
-----|-----------|----------|----------
  3   |   118.91   |     0.00   | Exercise
  3   |   105.94   |     0.00   | Exercise
  3   |    94.39   |     0.61   | Exercise
  3   |    84.10   |    10.90   | Exercise
-----|-----------|----------|----------
```

对于一般的美式期权问题时间复杂度(即对 T 步的美式期权)

基础算法：$O(T^2)$

每层 t 需计算 t+1 个节点，总操作数为$\sum_{t=1}^{T} t = \frac{T(T+1)}{2}$

上述例子是将实际问题大大简化后得到的，对更高维更复杂的问题，也可加以推广。